

Day 3 — Navigation, Overlays, and SPA Patterns

Front-End Accessibility • Day 3 of 3

Keep Users Oriented

Accessible apps must always tell users where they are and what is on screen.

Communicate State Changes

Every content update, disclosure, and route change must be announced to assistive technology.

Manage Focus Predictably

Focus must move in a logical, expected sequence — especially across dialogs, overlays, and SPA views.

- 📄 Trainer: Welcome participants to Day 3. Remind them that Days 1 and 2 built the foundation — today they put it into practice with real interaction patterns. Ask: "When you last used a screen reader on a single-page app, did you know when the view had changed?" Connect their answer to the three principles on screen. No code files needed yet.

Today's Learning Approach

Every exercise today follows the same four-step cycle. This mirrors the workflow used by accessibility engineers in production teams — audit first, inspect second, fix with intent, then validate with a real screen reader.

1

1 • Test the Experience

Use NVDA or JAWS before touching any code. Let the screen reader reveal what is broken.

2

2 • Inspect the Code

Open DevTools and the source file. Look at the rendered HTML, not just the authored markup.

3

3 • Fix Interaction, State, and Focus

Address the root cause — semantic role, ARIA state, keyboard handler, or focus management.

4

4 • Retest with NVDA or JAWS

Confirm the fix resolves the issue. A passing visual check is not enough.

- ❏ Trainer: Emphasise the retest step — many developers consider an accessibility fix done when it looks correct visually. Explain that NVDA and JAWS are the ground truth for today. Ask: "What is the risk of only testing with a mouse?" Connect to how aria-expanded or focus placement cannot be verified visually.

Days 1 & 2 Recap

Before diving into today's interaction patterns, it is worth anchoring where we have come from. Each day builds on the last — today's work only makes sense with a solid semantic and form foundation beneath it.

Day 1 • Semantic Structure

Headings, landmarks, lists, and the accessibility tree. Building meaning into HTML so assistive technology can navigate it.

1

2

3

Day 3 • Interaction & Orientation

Navigation patterns, disclosure widgets, dialogs, SPA route changes, and responsive menus. The focus of today's full session.

Day 2 • Form Relationships

Labels, error messages, required fields, and grouping. Ensuring every form control has a name, role, and value that a screen reader can expose.

- ❏ Trainer: Spend no more than three minutes on this slide. Ask: "What is one thing from Day 2 that surprised you?" Use the answer to remind participants that ARIA states like `aria-invalid` and `aria-describedby` follow the same pattern as `aria-expanded` and `aria-current` introduced today. This connects prior learning to new patterns.

What You Will Be Able to Do by End of Day

These outcomes are practical and verifiable. By the end of today, you should be able to open any interactive page and systematically audit it for orientation, state, and focus issues using a screen reader alongside DevTools.

1

Build Accessible Navigation Patterns

Apply `aria-current`, landmark roles, and meaningful link text to navigation regions.

2

Test Accordions, Dialogs, and Overlays

Use NVDA and JAWS to verify `aria-expanded`, focus movement, and background inertness.

3

Understand SPA Focus and Route-Change Behaviour

Know what React and Angular must do when a view changes, and how to verify it with a screen reader.

- 📋 Trainer: Read each outcome aloud and ask participants to rate their current confidence on a scale of one to five. This sets a baseline for the day and surfaces any gaps in prior knowledge. No code files needed yet. Mention that all three outcomes will be practised through the six exercise files introduced throughout the day.

The Core Day 3 Question

Every interaction pattern covered today — navigation, accordions, dialogs, SPA views — can be evaluated against three fundamental questions. If a screen reader user cannot answer all three confidently, the implementation needs work.

Where am I?

Does the page or view communicate its location through a meaningful title, heading, and landmark structure?

What changed?

When content is expanded, collapsed, loaded, or updated — is that change announced to assistive technology?

Where did focus go?

After a user action — opening a dialog, navigating a route, or closing an overlay — does focus land somewhere logical and expected?

- 📄 Trainer: This is a key orientation slide. Encourage participants to write these three questions down — they serve as a personal audit checklist for the rest of the day. Ask: "Think of a broken interaction you have encountered. Which of these three questions did it fail?" Return to these questions at the end of each exercise to reinforce the framework.

Navigation and Orientation

Navigation is not simply a list of links. For screen reader users, a well-structured navigation region is a primary wayfinding tool — it tells them where the site is organised, where they currently are, and where they can go next. Getting this wrong means users spend significant time exploring a page just to understand its structure.

- Navigation regions give structure to the page at a landmark level
- Link text must describe the destination, not the action (avoid "click here")
- The current page or section must be exposed in code — not just visually styled
- Multiple navigation regions must each have a distinct accessible label

Key Principle

Visual styling alone — such as a bold or underlined active link — is invisible to screen reader users. The current location must be communicated through code, not colour.

- 📋 Trainer: Ask participants to open a production site of their choice and navigate to its main menu using only the keyboard. Ask: "Can you tell which link represents the current page without looking at the URL bar?" Connect their finding to the need for `aria-current="page"`. No exercise file yet — this is conceptual setup for the navigation exercise on the next few slides.

Main Navigation Pattern

A well-implemented main navigation region uses the correct HTML element, provides a distinct label, expresses meaningful link text, and marks the current page programmatically. These four requirements are the baseline for any navigation component — including those built in React or Angular.

Use `nav` for Navigation Regions

The `<nav>` element creates a navigation landmark. Multiple `<nav>` elements on the same page must each have a unique `aria-label` so screen readers can distinguish them (e.g. "Main", "Breadcrumb", "Footer").

Use Meaningful Link Text

Every link must describe its destination in context. Avoid generic text such as "Read more" or "Here". If context is provided by surrounding text, use `aria-label` or `aria-labelledby` to supplement.

Use `aria-current` for the Current Page

Apply `aria-current="page"` to the link that matches the current URL. In SPAs, this attribute must be updated dynamically as routes change — it is not set once at build time.

- ❏ Trainer: Show a live code snippet of a `nav` element with and without `aria-current`. Demonstrate in NVDA how the virtual cursor reads "current page" when the attribute is present. Ask: "In your React or Angular component, where would you set and update `aria-current`?" Reference `react-navigation-pattern.jsx` as a worked example of this pattern.

Breadcrumbs

Breadcrumbs serve a specific orientation function — they show a user where they are within a site hierarchy and allow them to backtrack. For screen reader users, a breadcrumb that is not marked up correctly is indistinguishable from any other list of links. Proper implementation makes the breadcrumb's purpose explicit.

1 Use `nav` `aria-label="Breadcrumb"`

Wrap the breadcrumb in a `<nav>` element with `aria-label="Breadcrumb"`. This creates a named landmark that screen reader users can jump to directly from the landmarks menu.

2 Use an Ordered List

Breadcrumb items represent a sequence — use `<ol>` so that screen readers announce the item count and position (e.g. "item 2 of 3").

3 Mark the Current Page Clearly

Apply `aria-current="page"` to the final breadcrumb item. It may or may not be a link — if it is not interactive, use a `<span>` rather than an `<a>`.

- 📋 Trainer: Demonstrate the difference between a breadcrumb with and without `aria-label` on the `nav` element using JAWS landmarks list. Ask: "Why does it matter whether the current breadcrumb item is a link or not?" Connect to the principle that non-interactive elements should not be focusable. No exercise file for this slide — breadcrumb is covered within 01-bad-navigation.html.

EXERCISE 01

Exercise: Navigation Audit

This exercise develops systematic audit skills for navigation regions. You will move from a broken implementation to a corrected one, using a screen reader to validate each change. The goal is to identify exactly what NVDA or JAWS cannot determine from the broken file, and to confirm that the good file resolves each gap.

01

Open 01-bad-navigation.html

Use NVDA or JAWS. Navigate using the landmarks list, headings list, and Tab key. Note what is missing or confusing.

03

Compare with 02-good-navigation.html

Inspect the corrected file in DevTools. Identify each specific code change — `<nav>`, `aria-label`, `aria-current`, and meaningful link text.

02

Work Through the Exercise Sheet

Open `exercise-01-navigation-and-orientation-audit.md`. Follow the audit checklist: test headings, landmarks, link text, and current page state.

04

Discuss Findings

Share one thing NVDA or JAWS could not tell you from the bad file, and confirm the good file resolves it.

- ❏ Trainer: Give participants 12–15 minutes for this exercise. Open `01-bad-navigation.html` yourself and demo the landmarks menu in NVDA before participants begin. Ask: "What did JAWS or NVDA say when you reached the active link in the bad file?" Connect the answer directly to the absence of `aria-current`. Reference `react-navigation-pattern.jsx` if any participant uses React.

Accordions and Disclosures

An accordion or disclosure widget allows users to show and hide sections of content. For keyboard and screen reader users, the control mechanism must be a real button, and the current state of that button — expanded or collapsed — must be communicated through code, not just through visual changes like a rotating chevron.

When content is hidden, it must be truly hidden — not just visually off-screen — so that keyboard users cannot tab into it and screen reader users do not encounter it in the reading order.

Three Non-Negotiables

- Use a real `<button>` element — not a `<div>` or `<span>`
- Use `aria-expanded` to communicate open or closed state
- Use `display: none` or `hidden` to remove collapsed content from the accessibility tree

📋 Trainer: Ask: "What happens when a screen reader encounters a div with an onclick handler instead of a button?" Demonstrate with DevTools how a `<div>` has no implicit role and no keyboard access. Connect to how React developers sometimes use `onClick` on non-interactive elements — this is where `react-dialog-pattern.jsx` shows the correct button-based approach. No exercise file yet — this is setup for exercise 02.

Accordion Implementation Rules

Implementing an accordion correctly requires attention to three connected concerns: the button, the panel it controls, and the JavaScript that manages both. All three must work together — fixing only one or two of them will leave the pattern partially broken for screen reader users.

1

Button Controls the Panel

The trigger must be a `<button>`. It gets keyboard focus automatically, can be activated with Enter or Space, and exposes its role to assistive technology without additional ARIA.

2

`aria-controls` Connects Button to Panel

Set `aria-controls` on the button to reference the id of its associated panel. This allows screen readers to navigate directly from the button to the controlled region.

3

JavaScript Must Update Visibility and State

On toggle, JavaScript must: set `aria-expanded` to true or false on the button, and show or hide the panel using `display: none` or the `hidden` attribute — not CSS opacity or transform alone.

- ❏ Trainer: Open DevTools and show the accordion button element live. Demonstrate that `aria-expanded` changes value in the DOM when the panel opens. Ask: "What would a screen reader announce if `aria-expanded` was never updated?" Connect the answer to how JAWS reads button state. Reference `react-dialog-pattern.jsx` for a React example of state-driven ARIA attribute updates.

Exercise: Accordion Disclosure Pattern

This exercise focuses on diagnosing accordion failures using a screen reader, then verifying the corrected implementation. Pay particular attention to what NVDA or JAWS announces when a button is focused and when a panel opens or closes.

Open 03-bad-accordion.html

Navigate using Tab and Enter. Note what the screen reader announces when you activate the toggle. Does it say "expanded" or "collapsed"? Can you reach the panel content with Tab?

Work Through exercise-02-accordion-disclosure-pattern.md

Follow the checklist: check the element type, inspect aria-expanded, verify panel visibility method, and test keyboard behaviour.

Compare with 04-good-accordion.html

Open the corrected file. Inspect the DOM in DevTools as you open and close panels. Confirm that aria-expanded updates and that the panel is genuinely hidden when collapsed.

- 📄 Trainer: Demo the bad file first — navigate to the accordion trigger with Tab and activate it. Ask: "What did NVDA say?" If it said nothing about state, that is the bug. After the exercise, ask: "Where in a React component would you update aria-expanded?" Connect to state management — aria-expanded should be driven by component state, not a separate DOM manipulation.

Dialogs and Modals

A dialog is a significant shift in the interaction context. Unlike an accordion, which reveals content inline, a dialog overlays the page and demands immediate attention. Screen reader users must be transported into the dialog immediately upon opening, and the rest of the page must become unreachable until the dialog is closed. Getting either of these wrong creates a fundamentally broken experience.



Focus Must Move Into the Dialog

When the dialog opens, focus must move to either the dialog container, its heading, or its first focusable element. Without this, screen reader users remain on the page and must search for the dialog manually.



Background Must Become Inert

All content outside the dialog must be made unreachable — via Tab, arrow keys, and screen reader virtual cursor. The `inert` attribute or a focus trap achieves this reliably.



Focus Must Return After Closing

When the dialog closes, focus must return to the element that triggered it — typically the button that opened the dialog. Without this, keyboard users lose their place entirely.

- ❏ Trainer: Ask: "What happens if a user opens a dialog and focus does not move?" Demonstrate with NVDA on 05-bad-dialog.html — the user continues reading the background page. This is a critical failure. Connect to how React's `useRef` and `useEffect` are used in `react-dialog-pattern.jsx` to move focus programmatically when the dialog mounts.

Accessible Dialog Requirements

An accessible dialog implementation requires four specific technical conditions to be met. Each one addresses a distinct failure mode that screen reader users will encounter if it is missing. All four must be present — partial compliance still produces a broken experience.

Dialog Needs an Accessible Name

Use `role="dialog"` and `aria-labelledby` referencing the dialog's visible heading, or `aria-label` if no visible heading exists. Without a name, screen readers announce only "dialog" with no context.

Background Content Must Not Be Reachable

Apply `inert` to all sibling elements outside the dialog, or implement a focus trap using JavaScript. Test with Tab and with NVDA's Browse mode arrow keys — both must be contained.

Escape and Close Controls Must Work Predictably

The Escape key must close the dialog and return focus to the trigger. A visible close button must also be present inside the dialog, labelled with "Close" or "Close [dialog name]".

- ❏ Trainer: Open DevTools and inspect a dialog element. Show the difference between a dialog with `aria-labelledby` and one without — demonstrate what JAWS announces in each case. Ask: "How would you implement focus trap logic in a React functional component?" Reference `react-dialog-pattern.jsx` which demonstrates focus trap using `useRef` and a `keydown` handler.

Exercise: Dialog Focus Management

Focus management in dialogs is one of the most frequently broken patterns in production applications. This exercise asks you to experience the failure before reading the fix — so that the solution has concrete meaning grounded in the screen reader experience.

01

Open **05-bad-dialog.html**

Open the dialog using the trigger button. Note where NVDA or JAWS places focus. Try tabbing — can you reach content behind the dialog? Press Escape — does focus return to the trigger?

02

Work Through **exercise-03-dialog-focus-management.md**

Follow the checklist: test focus entry, background inertness, Escape key behaviour, and focus return. Document each failure with the specific screen reader announcement you heard.

03

Compare with **06-good-dialog.html**

Open the corrected file. Repeat each test. Inspect **role**, **aria-labelledby**, **inert**, and focus management code in DevTools.

- 📄 Trainer: Allow 15 minutes. Demo the bad file first with NVDA — open the dialog and show that focus does not move. Ask: "What would a screen reader user do next if focus didn't move?" The answer (search the whole page) highlights how disorienting this failure is. After comparing files, ask: "Where in react-dialog-pattern.jsx does focus get moved into the dialog?"

SPA Route Changes

In a traditional multi-page website, every navigation action causes a full page reload. The browser announces the new page title, moves focus to the top of the document, and resets the screen reader's position. In a React or Angular single-page application, none of this happens automatically — the URL changes, content updates, but the screen reader has no signal that anything changed at all.

What the Browser Does Automatically

- Announces the new page title
- Moves focus to the top of the document
- Resets scroll position

What SPAs Must Do Manually

- Update `document.title` to reflect the new view
- Move focus to the main heading or `<main>` region
- Update `aria-current` in navigation

📄 Trainer: Ask: "Has anyone used NVDA on a React application and found themselves reading the same content after clicking a link?" This is the classic SPA route-change failure. Open `07-spa-route-change-bad.html` and demonstrate — change the view and ask participants what NVDA announced. Reference `angular-route-focus-example.ts` as the Angular implementation approach for this problem.

SPA Accessibility Responsibilities

When a SPA route changes, the application must replicate what the browser does automatically in a traditional page load. These three responsibilities apply to every framework — React, Angular, Vue, or otherwise. They are not framework-specific — they are accessibility requirements that the framework does not satisfy for you.



Update the Document Title

Set `document.title` to reflect the current view on every route change. The title is the first thing a screen reader announces when a page loads — in an SPA, it must be updated manually in a route change handler.



Move Focus to the Main Heading or Main Region

After the new view renders, move focus programmatically to the page's `<h1>` or `<main>` element using `element.focus()`. The target element may need `tabindex="-1"` to accept programmatic focus.



Update Current Navigation State

Update `aria-current="page"` on the active navigation link after every route change. In React, this is typically managed in the router's active link component. In Angular, see [angular-route-focus-example.ts](#).

- ❏ Trainer: Open `react-navigation-pattern.jsx` and show how route change handlers update `document.title` and move focus. Then open `angular-route-focus-example.ts` and show the Angular Router events approach. Ask: "If you add `tabindex="-1"` to a heading to allow focus, what does that mean for keyboard navigation?" Confirm that `tabindex="-1"` allows programmatic focus but does not add the element to the Tab sequence.

Exercise: SPA Route-Change Focus

This exercise surfaces the disorientation a screen reader user experiences in an SPA that does not manage focus or state on route change. The contrast between the bad and good files should be striking — the good file should feel like a traditional page load from the screen reader's perspective.

Open 07-spa-route-change-bad.html

Switch between views using the navigation links. Note what NVDA or JAWS announces. Does the title change? Does focus move? Where are you after the switch?

Work Through exercise-04-spa-route-change-focus.md

Document what is missing: title update, focus management, and aria-current state. Propose a fix for each gap based on today's framework patterns.

Compare with 08-spa-route-change-good.html

Repeat the navigation test. Confirm the title updates, focus moves to the main heading, and the active link shows aria-current="page" in DevTools.

- ❏ Trainer: Give 12–15 minutes. Demo the bad file first — switch views with NVDA running and ask: "What did the screen reader say?" Silence or no announcement is the expected failure. After the exercise, connect the solution to react-navigation-pattern.jsx and angular-route-focus-example.ts. Ask: "Which of today's three core questions did this file fail?"

Responsive Navigation

On small screens, navigation menus are typically hidden behind a toggle button — often called a hamburger menu. This is fundamentally a disclosure pattern: a button reveals and conceals a set of links. The same rules that apply to accordions apply here — the button must communicate its state, and the hidden links must be truly hidden when the menu is collapsed.

Menu Buttons Are Disclosure Controls

The menu toggle button must use `aria-expanded="true"` when the menu is open and `aria-expanded="false"` when it is closed. The label should be "Menu" or "Navigation" — not an unlabelled icon button.

Hidden Links Must Be Truly Hidden

When the menu is collapsed, links inside it must not be reachable via Tab. Use `display: none`, the `hidden` attribute, or `visibility: hidden` — not only a CSS transform or opacity transition, which leaves links in the Tab order.

Focus Management on Open and Close

When the menu opens, focus may remain on the button or move to the first link depending on the pattern. When it closes, focus must return to the toggle button. The behaviour must be consistent and predictable.

- ❏ Trainer: Ask: "What does NVDA say when you Tab to a menu button that has no label and no `aria-expanded`?" Demonstrate on a poorly implemented example. Connect to how many icon-only buttons in React or Angular components lack accessible names. Reference `09-responsive-nav-tooltip-starter.html` as the exercise file for this topic. Ask: "How would you add an accessible name to an icon-only button in React?"

Tooltips and Help Text

Tooltips are a common pattern for surfacing additional context — but many implementations are only available on mouse hover, which excludes keyboard users entirely. For screen reader users, a tooltip that is not connected to its trigger via ARIA is completely invisible. The safest approach is to use visible help text rather than tooltips wherever the information is important.

The Problem with Hover-Only Tooltips

- Mouse-only — keyboard users cannot access the content
- Not announced by screen readers unless connected via ARIA
- Often disappear before they can be read
- Frequently fail WCAG 1.4.13 (Content on Hover or Focus)

The Accessible Approach

- Use visible help text for important information
- Connect supplementary text with `aria-describedby`
- Ensure tooltips also appear on keyboard focus, not only hover
- Persistent, dismissible, and hoverable (WCAG 1.4.13)

📋 Trainer: Ask: "If a form field has a tooltip with a password requirement, and that tooltip only appears on hover, what happens to a screen reader user?" The answer — they cannot access the requirement — motivates the `aria-describedby` pattern. Connect to Day 2 form labelling work. Reference [09-responsive-nav-tooltip-starter.html](#) which includes both the responsive nav and a tooltip for participants to review and improve.

Exercise: Responsive Nav and Tooltip Review

This exercise combines two patterns — the responsive navigation menu and contextual help text — in a single starter file. The goal is to apply today's disclosure and ARIA description principles together, then verify the result with a screen reader.

01

Open 09-responsive-nav-tooltip-starter.html

Test the menu toggle with Tab and Enter. Check `aria-expanded` in DevTools. Navigate with NVDA — can you Tab to links when the menu is closed? What does the screen reader say about the toggle button's state?

02

Fix the Menu Button State

Work through `exercise-05-responsive-nav-tooltip-review.md`. Add or correct `aria-expanded`, ensure hidden links are removed from the Tab order when collapsed, and verify the button has an accessible name.

03

Improve Tooltip or Help Text Behaviour

Identify any hover-only tooltip. Either convert it to visible help text or connect it properly using `aria-describedby`. Verify it is accessible to keyboard and screen reader users.

- 📋 Trainer: Give 15 minutes. Circulate and check participants' DevTools — look for `aria-expanded` toggling in the DOM and `aria-describedby` references. Ask: "After your fix, what does NVDA say when you activate the toggle button?" The expected answer should include "expanded" or "collapsed". Connect tooltip fix to `aria-describedby` covered in Day 2.

Framework Perspective: React and Angular

React and Angular are powerful tools for managing UI state, but they do not automatically manage accessibility state. Every ARIA attribute, focus movement, and screen reader announcement must be explicitly implemented by the developer. Understanding where and how to do this within each framework's lifecycle is essential for building accessible interactive components.



State Changes Must Update ARIA

When React or Angular state changes — dialog open, panel expanded, route changed — the corresponding ARIA attributes must update in the rendered HTML. Derive ARIA attributes from component state, not from separate DOM manipulation.



Components Must Preserve Keyboard Behaviour

Custom components must implement keyboard handlers explicitly. React and Angular do not add keyboard support to non-semantic elements. Use native elements where possible; add `onKeyDown` handlers where custom behaviour is required.



Route Changes Must Manage Focus

Router libraries do not move focus on view change. Implement a focus management strategy in your router configuration using lifecycle hooks — `useEffect` in React, `NavigationEnd` events in Angular.

- 📄 **Trainer:** Open `react-navigation-pattern.jsx` and `angular-route-focus-example.ts` side by side if possible. Show how each framework approaches route-change focus management differently, but the accessibility outcome is identical. Ask: "What is the risk of using a third-party component library that does not expose ARIA state?" Connect to the principle that rendered HTML is what assistive technology sees — not JSX or TypeScript.

React and Angular Reminders

These reminders apply across every exercise and every component you build. They are not advanced concepts — they are baseline requirements for any interactive component that users with disabilities will encounter. Apply them as a checklist when reviewing your own or a colleague's component.

→ State Changes Must Update ARIA

Bind `aria-expanded`, `aria-current`, `aria-hidden`, and `aria-modal` directly to component state variables. Never update ARIA attributes via direct DOM manipulation in a React or Angular component — let the framework's rendering cycle handle it.

→ Components Must Preserve Keyboard Behaviour

If you use a non-semantic element, you must add `role`, `tabindex="0"`, and keyboard event handlers explicitly. Test every interactive component with Tab, Enter, Space, and Escape before considering it complete.

→ Route Changes Must Manage Focus Intentionally

In React, use `useEffect` with the route as a dependency to move focus after navigation. In Angular, subscribe to `NavigationEnd` in a root service. Reference `angular-route-focus-example.ts` and `react-navigation-pattern.jsx` for implementation patterns.

- ☐ Trainer: Use this slide as a consolidation checkpoint before the final exercise. Ask participants to close their laptops briefly and recall the three reminders from memory. Then ask: "Which of these three did you encounter most in today's exercises?" Connect their answer to the pattern most relevant to their own project or codebase. Reference both framework files as take-home reference material.

Final Mini Challenge

This challenge brings together everything from Day 3. You will work with a single interactive page that contains navigation, an accordion, a dialog, and a simulated SPA route change — all with accessibility failures. Your task is to identify and fix as many issues as possible within the time available, then compare your work against the provided solution file.

Open the Starter

Open `10-final-interactive-page-starter.html` and run through it with NVDA or JAWS. Use the three core questions: Where am I? What changed? Where did focus go?

1

Retest with Screen Reader

After each fix, retest with NVDA or JAWS. Do not rely on visual or DevTools inspection alone. Each fix must produce a measurable improvement in the screen reader experience.

2

3

4

Audit and Fix

Work through `exercise-06-final-interactive-page-challenge.md`. Address navigation, accordion, dialog, and route-change patterns. Prioritise the most critical failures first.

Compare with Solution

Open `11-final-interactive-page-solution.html`. Inspect the code differences. Note any issue you missed and understand why each fix was made.

- ❏ Trainer: Allow 20–25 minutes for this exercise — it is the most substantial of the day. Circulate actively. Prioritise helping participants who are stuck on focus management or ARIA state. After the exercise, run a group debrief: "What was the hardest issue to find? What was the hardest to fix?" Connect both answers back to the three core questions and the framework files as ongoing references.

Day 3 Key Takeaway

Accessible interaction is not a single feature or a checklist item — it is an orientation system that keeps every user informed and in control at all times. When content changes, users must know. When focus moves, it must land somewhere logical. When a view updates, the application must take responsibility for that transition.

Manage structure, state, focus, and announcements together — not separately. A fix that addresses only one of these four concerns will leave the others broken.

Structure

Landmarks, headings, and semantic HTML give users a map of the page.

State

ARIA attributes communicate what is expanded, selected, current, or invalid.

Focus

Programmatic focus management keeps users oriented after every interaction.

Announcements

Screen reader announcements confirm that changes have happened and what they mean.

- ❏ Trainer: End the day by returning to the three core questions from slide 5. Ask each participant to name one thing they will change in their own codebase as a result of today. Thank participants for their engagement. Remind them that all exercise files, solution files, and framework reference patterns — including `react-navigation-pattern.jsx`, `react-dialog-pattern.jsx`, and `angular-route-focus-example.ts` — are available to take away. Day 3 complete.